POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

A lightweight canonicalization engine in C

WILLIAM ZHANG
Département de Electrical Engineering

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Electrical Engineering

July 2026

© William Zhang, 2026.

POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

Ce mémoire intitulé :

A lightweight canonicalization engine in C

présenté par **William ZHANG**
en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

Danielle DUBOIS, présidente
Sébastien VIGOT, membre et directeur de recherche
Marie COUTURE, membre et codirectrice de recherche
Jean TREMBLAY, membre
Joseph BROWN, membre externe

DEDICATION

À tous mes amis du labos,
vous me manquerez...

ACKNOWLEDGEMENTS

Texte / Text.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE OF CONTENTS

DEDICATION iii

ACKNOWLEDGEMENTS iv

RÉSUMÉ v

ABSTRACT vi

LIST OF TABLES ix

LIST OF FIGURES x

LIST OF SYMBOLS AND ACRONYMS xi

LIST OF APPENDICES xii

CHAPTER 1 INTRODUCTION 1
- 1.1 Definitions and basic concepts 1
- 1.2 Problem statement 1
- 1.3 Research objectives 1
- 1.4 Thesis outline 1

CHAPTER 2 LITERATURE REVIEW 2
- 2.1 Historical Background 2
- 2.2 Related work 3
  - 2.2.1 `JuMP` and `MathOptInterface` 4
  - 2.2.2 `CVXPY` 5
- 2.3 Canonicalization and Coefficient Extraction 6
  - 2.3.1 A worked example: least-squares 6
  - 2.3.2 A worked example: semidefinite program 8
  - 2.3.3 Expression trees and linear operators 10

CHAPTER 3 METHODOLOGY 12
- 3.1 Current approach and its limitations 12
- 3.2 Architecture overview 13
- 3.3 Affine atoms 13
  - 3.3.1 Direct encoding of linear operators 14

3.3.2 The `sum` atom and the `idx_map` trick 14
3.3.3 The `left_matmul` atom and Kronecker avoidance 16
3.3.4 The `index` atom and structural row selection 17
3.4 Optimizations 18
3.5 Parameter handling 18

CHAPTER 4 BENCHMARK RESULTS 19

CHAPTER 5 Benchmark results 20

CHAPTER 6 CONCLUSION 21
6.1 Synthèse des travaux / Summary of Works 21
6.2 Limitations de la solution proposée / Limitations 21
6.3 Améliorations futures / Future Research 21

REFERENCES 22

APPENDICES 26

LIST OF TABLES

Table 3.1 Catalogue of affine atoms. Each row describes one node type, its mathematical operation, and the core CSR transformation applied to the child's Jacobian during `eval_jacobian`. 13

LIST OF FIGURES

Figure 2.1 Expression tree for the affine expression `sum(A @ x - b) + C @ y`. . 10

Figure 3.1 Coefficient matrix accumulation for `sum(A @ x)`. Each node composes its operator with the child's coefficient matrix via sparse matrix multiplication. 12

Figure 3.2 CSR representation before and after applying `sum`. The 3-row matrix collapses to 1 row by setting `indptr` $= [0, 6]$. Duplicate column indices (0 and 2) represent values that sum together. The `indices` and `data` arrays remain unchanged—no matrix multiplication required. 14

LIST OF SYMBOLS AND ACRONYMS

IETF	Internet Engineering Task Force
OSI	Open Systems Interconnection

LIST OF APPENDICES

Appendix A Démo 26
Appendix B Encore une annexe / Another Appendix 27
Appendix C Une dernière annexe / The Last Appendix 28

CHAPTER 1 INTRODUCTION

1.1 Definitions and basic concepts

1.2 Problem statement

1.3 Research objectives

1.4 Thesis outline

CHAPTER 2 LITERATURE REVIEW

2.1 Historical Background

Solving an optimization problem on a computer requires translating the mathematical formulation into a data format that solvers can process. For linear programs, this means constructing the constraint matrix A, the right-hand side vector b, and the objective coefficients c that define the *standard form*,

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned} \tag{2.1}$$

Manual encoding In the early days of linear programming, analysts manually computed and encoded each coefficient onto punch cards. This process was tedious, error-prone and became impractical as linear programming found more and more applications in large-scale industrial problems [1].

MPS format IBM's Mathematical Programming System (MPS) file format, developed in the 1960s, established a standardized file format for representing linear programs [2]. The MPS format uses fixed-width fields designed for 80-column punch cards, with sections for rows, columns, and coefficients. William Orchard-Hays [3] provides a comprehensive history of these early mathematical programming systems, tracing their development from 1952 at RAND Corporation through their widespread adoption on mainframe computers in the late 1960s.

Matrix generators To automate the creation of MPS files, *matrix generators* emerged in the late 1960s; they are procedural programs that compute the problem coefficients through explicit loops and output the results in MPS format [4]. Commercial systems such as `MaGen` [5], `GAMMA` [6], and `DATAFORM` [7] were among the first matrix generators which were used in various domains.

A modern approach Modelers naturally think in terms of sets, indices, and algebraic expressions, not in terms of individual matrix entries. Matrix generators simplified the encoding process but still required modelers to translate their mathematical formulations into

procedural code that explicitly constructs the coefficient matrices. This suggests that the gap between the modeler's representation of the problem and the solver's required format hadn't been resolved yet [1].

Algebraic modeling languages In the late 1970s, *algebraic modeling languages* emerged as formal languages designed to accept problem descriptions that closely mirror standard mathematical notation [8]. By working directly with the modeler's representation, these languages simplify verification (the model can be inspected as written), modification (changing the problem does not require rewriting procedural code), and communication (the formulation serves as its own documentation). In the late 1980s and early 1990s, commercial algebraic modeling systems such as `GAMS` [9], `AIMMS` [10] and `AMPL` [11] became widely adopted in both academia and industry (and they are still widely used today).

2.2 Related work

In the 2000s, advances in convex optimization made it practical to solve a much broader class of problems efficiently. Solvers such as `MOSEK` [12, 13] adopted *conic optimization* as their interface—a structured subclass of convex optimization with the standard form [14]:

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & b - Ax \in \mathcal{K}, \end{array} \tag{2.2}$$

where $\mathcal{K}$ is a closed convex cone. This formulation generalizes linear programming (when $\mathcal{K}$ is the nonnegative orthant) to include second-order cone programs (SOCPs), semidefinite programs (SDPs) and more.

Embedded modeling languages In the late 2000s and early 2010s, modeling languages emerged that are embedded in general-purpose programming languages (in contrast with standalone systems like `AMPL` and `GAMS`). Examples include `YALMIP` [15] in MATLAB, `Pyomo` [16] in Python, `JuMP` [17] in Julia, and `CasADi` [18] in C++.

Disciplined convex programming In addition, there are also *domain specific languages* that implement *disciplined convex programming* (DCP), a set of composition rules that can automatically verify convexity [19, 20]. These include `CVX` [21] in MATLAB, `CVXPY` [22, 23] in Python, `CVXR` [24] in R, and `Convex.jl` [25] in Julia.

Modern open-source modeling languages At the time of writing, most modeling languages mentioned above continue to be developed and have active communities. In the remainder of the section, we focus our attention on two in particular: `JuMP` for the design ideas introduced by `MathOptInterface`, and `CVXPY` because it is the modeling language on which the present work builds.

2.2.1 `JuMP` and `MathOptInterface`

`JuMP` is an algebraic modeling language embedded in Julia, originally developed in 2012 by Iain Dunning, Joey Huchette, and Miles Lubin at the MIT Operations Research Center [17, 26]. Its abstraction layer, `MathOptInterface` (`MOI`) [27], rethinks how a modeling language communicates with solvers, moving away from the traditional approach of reducing every problem to a single standard form.

Function-set pairs `MOI` represents each constraint as a *function-set pair*: a mathematical function paired with the set it must belong to. For example, a linear inequality $a^T x \leq b$ is encoded as:

```
MOI.add_constraint(model,
    MOI.ScalarAffineFunction(MOI.ScalarAffineTerm.(a, x), 0.0),
    MOI.LessThan(b))
```

while a second-order cone constraint $t \geq \|x\|_2$ with n variables uses:

```
MOI.add_constraint(model,
    MOI.VectorOfVariables([t; x]),
    MOI.SecondOrderCone(n)).
```

`MOI` therefore preserves the semantic type of each constraint, so solvers that accept specific constraint types can receive them directly without unnecessary reformulations.

Bridges When a solver does not natively support a particular function-set pair, `MOI` applies a *bridge*, a composable transformation to an equivalent representation the solver accepts. The bridge system maintains a directed hyper-graph of known transformations and finds the shortest path to the target solver's supported types, so adding a reformulation once benefits all solvers.

Incremental problem construction Some solvers, notably `MOSEK` [12] and `HiGHS` [28], allow constraints to be added one at a time rather than requiring the entire problem to

be submitted at once. `MOI` supports this incremental mode natively, so algorithms that iteratively refine a formulation, such as cutting-plane or branch-and-cut methods, can append a constraint between successive solves without reconstructing the full problem data.

Extensibility `MOI` makes it straightforward to support new constraint types: a solver declares which function-set pairs it handles, and new bridges can be registered at any time. `Hypatia` [29, 30], for example, natively implements over twenty specialized cones, including the log-determinant cone, the relative-entropy cone, and several spectral matrix cones, all of which are first-class `MOI` set types. `SCS` [31,32] has similarly begun adding native spectral matrix cone support [33], bypassing the large semidefinite reformulations that would otherwise be required.

2.2.2 `CVXPY`

`CVXPY` is a domain-specific language for convex optimization embedded in Python, that was originally developed by Steven Diamond and Stephen Boyd at Stanford University [22]. Akshay Agrawal later redesigned the rewriting system in `CVXPY` [23], which enabled support for other disciplined languages [34–36], and the project has since grown through contributions from many others. Unlike `MOI`, which preserves constraint diversity, `CVXPY` channels all problems through a single compilation pipeline targeting one unified standard form.

Unified standard form `CVXPY` rewrites every problem into a conic standard form with a quadratic objective:

$$\begin{aligned} &\text{minimize} && \tfrac{1}{2}\, x^T P\, x + c^T x \\ &\text{subject to} && Ax + b \in \mathcal{K}, \end{aligned} \tag{2.3}$$

Because the rewriting system targets a unique form, solver interfaces only need to map from this single representation.

Parameters and efficient re-solving `CVXPY` provides first-class support for parameters via `cp.Parameter`. When a problem contains parameters, `CVXPY` compiles the structure once—sparsity pattern, cone types, variable ordering—and stores a mapping from parameter values to numerical coefficients. Subsequent solves only require evaluating this mapping and calling the solver, skipping the full canonicalization. The `cvxpygen` library [37] extends this by generating standalone C code embedding the compiled structure, enabling deployment without a Python runtime.

Applications of parametric optimization Many applications require solving a sequence of related optimization problems that differ only in their data. In Model Predictive Control (MPC), a controller solves an optimization problem at each timestep with the current system state as input. In parameter sweeps, we explore the behavior of a system across a range of parameter values. These problems share the same structure—only the numerical coefficients change—and we want to solve them efficiently without reconstructing the problem from scratch each time. First-class parameter support in `CVXPY` addresses this need by separating the problem structure from its numerical data.

2.3 Canonicalization and Coefficient Extraction

The previous sections described what modeling languages do at a high level: they accept mathematical formulations and produce solver-ready data. This section examines *how* that translation is performed, using two concrete examples—a least-squares problem and a semidefinite program—to illustrate the machinery behind `CVXPY` and `MOI`.

2.3.1 A worked example: least-squares

Consider the least-squares problem

$$\text{minimize} \quad \|Fx - g\|_2^2, \tag{2.4}$$

with concrete data

$$F = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}, \quad g = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix},$$

so that $F \in \mathbf{R}^{3\times 2}$ and $g \in \mathbf{R}^3$. This problem is small enough to trace every step of the canonicalization, yet it exercises the key mechanisms of both systems.

`CVXPY`'s canonicalization pipeline In `CVXPY`, the problem is written as:

```
import numpy as np, cvxpy as cp
F = np.array([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])
g = np.array([1.0, 2.0, 3.0])
x = cp.Variable(2)
prob = cp.Problem(cp.Minimize(cp.sum_squares(F @ x - g)))
```

Chu et al. [38] describe the general pipeline that systems like `CVXPY` follow to convert such a formulation into solver-ready data. The first two steps are common to all canonicalization paths:

1. **Expression tree.** `CVXPY` builds a tree whose root is `sum_squares`, with child `sub`$(-)$, which in turn has children `matmul`(F, x) and constant g.
2. **DCP verification.** The `sum_squares` atom is convex, and its argument $Fx - g$ is affine; by DCP composition rules, the overall expression is convex.

What happens next depends on the target solver's capabilities.

Quadratic objective path `CVXPY`'s default canonicalization for `sum_squares` introduces an auxiliary variable $t = Fx - g$ (as a linear equality constraint) and sets the objective to $t^T t = \|t\|_2^2$. `CVXPY`'s standard form (2.3) includes a P matrix precisely for this case. With $z = (x_1, x_2, t_1, t_2, t_3)$, the QP data is:

$$P = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 2 \end{pmatrix}, \quad c = 0, \tag{2.5}$$

i.e.$P = \mathbf{diag}(0_2, 2I_3)$—sparse, with no need to form $F^T F$. The equality constraint $t - Fx = -g$ gives:

$$\begin{pmatrix} -1 & -2 & 1 & 0 & 0 \\ -3 & -4 & 0 & 1 & 0 \\ -5 & -6 & 0 & 0 & 1 \end{pmatrix} z = \begin{pmatrix} -1 \\ -2 \\ -3 \end{pmatrix}. \tag{2.6}$$

Direct support for quadratic objectives in conic solvers is a relatively recent development; solvers such as `MOSEK` [12] and `Gurobi` [39] now accept them natively.

Epigraph reformulation For solvers limited to conic constraints, such as `ECOS` [40], `CVXPY` instead applies an epigraph reformulation. The `sum_squares` atom's canonicalization method introduces an auxiliary variable t and rewrites the objective as minimize t subject to $\|Fx - g\|_2^2 \leq t$, which is equivalent to the second-order cone constraint $\left\| \begin{smallmatrix} Fx-g \\ \frac{1}{2}(t-1) \end{smallmatrix} \right\|_2 \leq \frac{1}{2}(t+1)$. Chu et al. call the intermediate step of introducing a variable per subexpression "Smith form," followed by "relaxation" to inequalities. `CVXPY` then assembles the constraint into the

form $\mathcal{A}z + \beta \in \mathcal{K}$ where $z = (x_1, x_2, t)$, and $\mathcal{K}$ is a second-order cone. The objective becomes $c^T z$ with $c = (0, 0, 1)$, selecting the epigraph variable.

For our example, the solver-ready data is:

$$c = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad \mathcal{A} = \begin{pmatrix} 0 & 0 & \frac{1}{2} \\ 0 & 0 & -\frac{1}{2} \\ 1 & 2 & 0 \\ 3 & 4 & 0 \\ 5 & 6 & 0 \end{pmatrix}, \quad \beta = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \\ -1 \\ -2 \\ -3 \end{pmatrix}, \tag{2.7}$$

with $\mathcal{K} = \mathcal{Q}^5$ (second-order cone of dimension 5).

The two paths can be contrasted side by side:

- **QP path:** $z = (x, t) \in \mathbf{R}^5$, $P = \mathbf{diag}(0_2, 2I_3)$, equality constraint $t = Fx - g$.
- **SOC path:** $z = (x, s) \in \mathbf{R}^3$, $c = (0, 0, 1)$, SOC constraint $(s, Fx - g) \in \mathcal{Q}^4$.

The user writes the same four lines of code; `CVXPY` selects the appropriate canonicalization path depending on the target solver.

`JuMP/MOI` perspective In `JuMP`, writing `@objective(model, Min, sum((F*x - g).^2))` creates a *scalar quadratic function*. `MOI` represents the objective as a (`ScalarQuadraticFunction`, `MinSense`) pair. If the solver accepts quadratic objectives (*e.g.*`MOSEK`, `Gurobi`), `MOI` passes the coefficients directly—no reformulation is needed. If the solver only supports conic constraints, `MOI`'s bridge system activates: a *quadratic-to-SOC bridge* rewrites the quadratic objective into a second-order cone constraint, introducing an epigraph variable exactly as in the SOC path above. The same user code, with a different solver backend, triggers a different chain of bridges—all automatically.

2.3.2 A worked example: semidefinite program

Consider the problem of finding the maximum eigenvalue of a symmetric matrix $C \in \mathbf{S}^2$:

$$\begin{array}{ll} \text{minimize} & -t \\ \text{subject to} & tI - C \succeq 0, \end{array} \tag{2.8}$$

with concrete data

$$C = \begin{pmatrix} 4 & 1 \\ 1 & 2 \end{pmatrix}.$$

The optimal value is $-\lambda_{\max}(C) \approx -4.24$. This example exercises the semidefinite cone $\mathbf{S}_+^n$, which requires *vectorization*—a step that reveals another dimension of solver-specific adaptation.

`CVXPY` path In `CVXPY`:

```
import numpy as np, cvxpy as cp
C = np.array([[4.0, 1.0], [1.0, 2.0]])
t = cp.Variable()
prob = cp.Problem(cp.Minimize(-t),
                  [t * np.eye(2) - C >> 0])
```

DCP verification is straightforward: the constraint $tI - C \succeq 0$ is an affine expression constrained to the positive semidefinite cone, which is a valid DCP constraint.

Canonicalization maps the matrix constraint into the standard form $\mathcal{A}z+\beta \in \mathcal{K}$ with $\mathcal{K} = \mathbf{S}_+^2$. The 2×2 matrix $tI - C$ is vectorized into a vector in $\mathbf{R}^3$ using *scaled lower-triangular* (svec) ordering:

$$\operatorname{svec}(tI - C) = \begin{pmatrix} t-4 \\ \sqrt{2}(0-1) \\ t-2 \end{pmatrix} = \begin{pmatrix} t-4 \\ -\sqrt{2} \\ t-2 \end{pmatrix}.$$

With $z = (t)$, the coefficient data is:

$$c = (-1), \quad \mathcal{A} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}, \quad \beta = \begin{pmatrix} -4 \\ -\sqrt{2} \\ -2 \end{pmatrix}, \tag{2.9}$$

with $\mathcal{K} = \mathbf{S}_+^2$ (PSD cone of order 2, vectorized to dimension 3).

`MOI/JuMP` path `MOI` represents this constraint as a function-set pair (`VectorAffineFunction`, `PositiveSemidefiniteConeTriangle`). The key difference from `CVXPY` is that different solvers expect different PSD formats:

- `MOSEK` expects scaled lower-triangular (svec) ordering, with off-diagonal entries scaled by $\sqrt{2}$.
- `SCS` expects the vectorized upper triangle without scaling.

`MOI` bridges automatically convert between these representations. The user writes one formulation; bridges adapt to each solver's expected format.

Contrast This mirrors the least-squares example: `MOI` preserves the semantic PSD constraint type and adapts per solver via bridges, while `CVXPY` flattens into one standard form with $\mathcal{K}$ containing a PSD cone. Both achieve solver-ready data through different architectural choices—the same principle of automatic adaptation, applied through different mechanisms.

2.3.3 Expression trees and linear operators

The worked examples above showed the *result* of canonicalization—the solver-ready coefficient data. We now examine the *representation* that modeling languages use internally to produce those coefficients.

`CVXPY`'s canonicalization guarantees that all constraints and the objective function are affine functions of the variable x. An affine function can be expressed as the composition of linear functions and addition of constants. `CVXPY` represents each affine function as an *expression tree*, where each node is a linear function or *atom*.

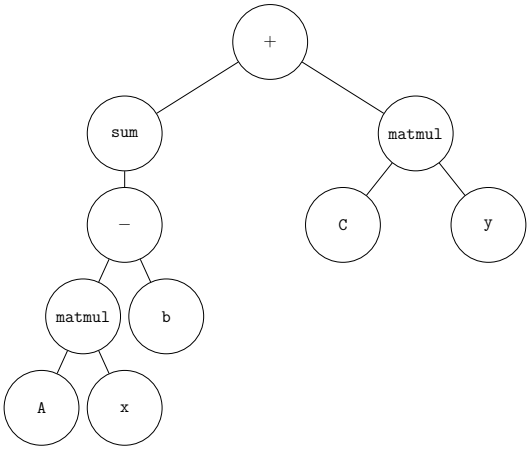


Figure 2.1 Expression tree for the affine expression `sum(A @ x - b) + C @ y`.

Column-major vectorization When a modeling language handles matrix variables, it must flatten them into vectors to build coefficient matrices. `CVXPY` uses column-major (Fortran) ordering: a matrix variable $X \in \mathbf{R}^{m \times n}$ is vectorized as $(X_{11}, X_{21}, \ldots, X_{m1}, X_{12}, \ldots, X_{mn})^T \in \mathbf{R}^{mn}$. This convention determines the column ordering in all coefficient matrices produced during canonicalization.

Composition of linear operators The power of the expression tree representation is that complex affine expressions reduce to compositions of simple operators. Consider the expression `cp.sum(A @ x + b)`, where $A \in \mathbf{R}^{5 \times 3}$ and $b \in \mathbf{R}^5$:

- the matrix multiplication operator $A \in \mathbf{R}^{5\times 3}$,
- the vector addition operator $+b$ where $b \in \mathbf{R}^5$,
- the sum operator $\mathbf{1}^T \in \mathbf{R}^{1\times 5}$.

The constant b is incorporated by augmenting the matrix with a column for the constant term. The resulting linear operator is $\mathbf{1}^T[A \mid b] \in \mathbf{R}^{1\times 4}$:

$$\underbrace{\begin{bmatrix}1 & 1 & 1 & 1 & 1\end{bmatrix}}_{\mathbf{1}^T}\underbrace{\left[\begin{array}{ccc|c} A_{11} & A_{12} & A_{13} & b_1 \\ A_{21} & A_{22} & A_{23} & b_2 \\ A_{31} & A_{32} & A_{33} & b_3 \\ A_{41} & A_{42} & A_{43} & b_4 \\ A_{51} & A_{52} & A_{53} & b_5 \end{array}\right]}_{[A|b]} = \begin{array}{cccc} x_1 & x_2 & x_3 & 1 \end{array}\left[\begin{array}{ccc|c} \sum_i A_{i1} & \sum_i A_{i2} & \sum_i A_{i3} & \sum_i b_i \end{array}\right]$$

Multi-variable problems When a problem involves multiple variables, `CVXPY` concatenates them into a single vector and each constraint produces one block-row of the global coefficient matrix. For example, with variables x (scalar), $y \in \mathbf{R}^3$, and $z \in \mathbf{R}^2$, the combined variable vector is $[x, y_1, y_2, y_3, z_1, z_2]^T$ and constraints such as `8*x == 28`, `cp.sum(y) + 5*cp.sum(z) == 58`, and `z == 26` are each converted to the form $Ax + b = 0$ and stacked into a single coefficient matrix:

$$\begin{array}{cccccc} x & y_1 & y_2 & y_3 & z_1 & z_2 \end{array}\quad 1$$
$$\left[\begin{array}{cccccc|c} 8 & 0 & 0 & 0 & 0 & 0 & -28 \\ 0 & 1 & 1 & 1 & 5 & 5 & -58 \\ 0 & 0 & 0 & 0 & 1 & 0 & -26 \\ 0 & 0 & 0 & 0 & 0 & 1 & -26 \end{array}\right]$$

Each row corresponds to one scalar equality; the columns are determined by the global variable ordering and the final column holds the constant term. The next chapter examines how this coefficient matrix is actually computed.

CHAPTER 3 METHODOLOGY

3.1 Current approach and its limitations

The previous chapter described expression trees and how `CVXPY` represents affine functions as compositions of linear operators. This section examines how `CVXPY` currently evaluates those trees to produce solver-ready coefficient matrices, and why that approach has limitations that motivate the present work.

Sparse matrix multiplication `CVXPY`'s current approach is to convert the linear operator tree into a single sparse matrix. The tree is traversed in post-order: each leaf node returns its coefficient matrix, and each internal node composes its children's matrices via sparse matrix multiplication. For example, in the expression `sum(A @ x)`, the traversal first returns A at the `matmul` node, then multiplies by $\mathbf{1}^T$ at the `sum` node, yielding $\mathbf{1}^T A$. The final matrix at the root is returned.

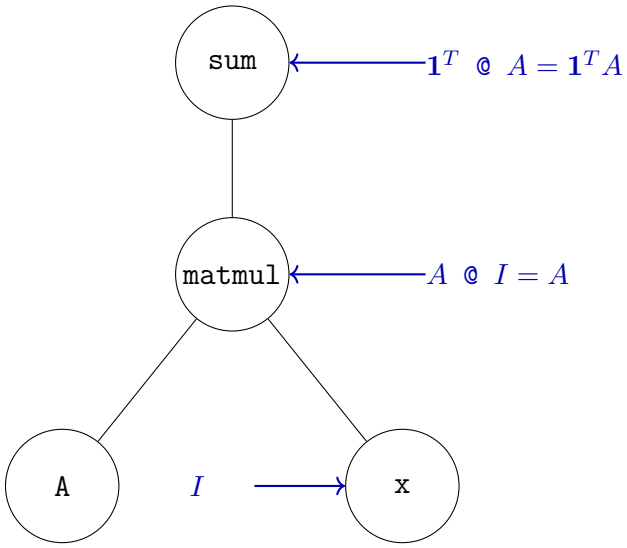


Figure 3.1 Coefficient matrix accumulation for `sum(A @ x)`. Each node composes its operator with the child's coefficient matrix via sparse matrix multiplication.

Limitations This approach requires storing intermediate sparse matrices at each node during traversal. For deep expression trees or large problem dimensions, this leads to significant memory overhead. Additionally, sparse matrix multiplication is computationally expensive

and may produce matrices with more nonzeros than the operands, further increasing memory and runtime costs.

Motivation These limitations are particularly acute in embedded and real-time settings where memory budgets are tight and latency requirements are strict. A method that avoids forming intermediate matrices entirely—by directly manipulating the sparse storage arrays—could achieve the same result with lower overhead. The remainder of this chapter develops such a method.

3.2 Architecture overview

3.3 Affine atoms

Table 3.1 catalogues the fifteen affine atoms implemented in the engine. Each atom exposes two entry points: `jacobian_init`, which determines the output Jacobian's sparsity pattern once at problem-build time, and `eval_jacobian`, which fills in the numerical values each time parameters change. The table's third column summarises the key CSR manipulation performed by each atom.

Table 3.1 Catalogue of affine atoms. Each row describes one node type, its mathematical operation, and the core CSR transformation applied to the child's Jacobian during `eval_jacobian`.

Atom	Math operation	Jacobian action on CSR
`variable`	$y = x$	Identity (leaf node)
`parameter`	$y = \theta$	None (data node)
`linear_op`	$y = Ax + b$	$J = A$ (stored directly)
`add`	$y = a + b$	Union sparsity, sum values
`neg`	$y = -x$	Same sparsity, negate `data`
`sum`	$y = \sum x$ (along axis)	Merge rows via `idx_map`
`hstack`	$y = [a; b; \ldots]$	Concatenate row blocks
`index`	$y = x[\text{idx}]$	Select/reorder rows
`reshape`	$\text{reshape}(x, m, n)$	No-op (copy arrays)
`transpose`	$y = X^T$	Permute rows
`broadcast`	$(1, n) \to (m, n)$	Tile/replicate rows
`promote`	scalar $\to$ vector	Replicate single row
`trace`	$y = \mathbf{Tr}(X)$	Sum diagonal rows via `idx_map`
`diag_vec`	$v \to \mathbf{diag}(v)$	Place rows at diagonal positions
`left_matmul`	$y = A \cdot f(x)$	Block-diagonal multiply (no Kronecker)

The subsections that follow first describe the general principle of encoding linear operators directly on CSR arrays, then examine three representative atoms in detail: `sum` (Section 3.3.2),

which illustrates the `idx_map` accumulation trick; `left_matmul` (Section 3.3.3), which avoids forming a Kronecker product; and `index` (Section 3.3.4), which performs pure structural row selection with no arithmetic.

3.3.1 Direct encoding of linear operators

Rather than composing operators via sparse matrix multiplication, we keep the coefficient matrix in CSR (Compressed Sparse Row) format and directly manipulate its index arrays without forming intermediate matrices. Each linear operator is implemented as a transformation on the CSR arrays `(indptr, indices, data)`. For example, the `sum` operator collapses all rows into row 0 by setting all entries of `indptr` to $[0, \texttt{nnz}]$, effectively mapping every row index to 0 without any matrix multiplication.

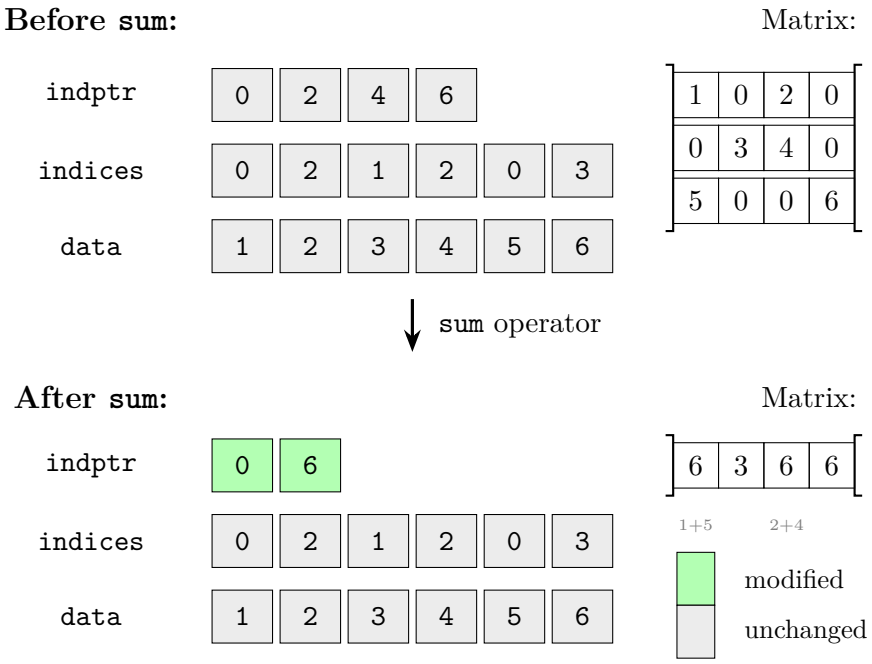


Figure 3.2 CSR representation before and after applying `sum`. The 3-row matrix collapses to 1 row by setting `indptr` $= [0, 6]$. Duplicate column indices (0 and 2) represent values that sum together. The `indices` and `data` arrays remain unchanged—no matrix multiplication required.

3.3.2 The `sum` atom and the `idx_map` trick

Summing along an axis collapses multiple rows of the child's Jacobian into fewer rows. Mathematically, for a full summation ($y = \mathbf{1}^T x$, scalar output), the Jacobian is $J_y = \mathbf{1}^T J_{\text{child}}$—every row of J_{child} is accumulated into a single output row. The naïve approach would compute

this via sparse matrix multiplication, allocating a new matrix each time. Instead, the engine precomputes a mapping from every nonzero position in the child's Jacobian to its target position in the output, then uses a simple scatter-accumulate loop at evaluation time.

Initialisation (Algorithm 1) During `jacobian_init`, the output sparsity pattern is determined by taking the union of all column indices that appear across the rows being summed. Duplicate column indices from different rows are merged into a single output entry. An array `idx_map` of length $\texttt{nnz}(J_{\text{child}})$ is then built: for each nonzero k in the child's Jacobian, `idx_map`$[k]$ stores the position in the output's `data` array where that value should be accumulated.

Algorithm 1 `sum` — Jacobian initialisation (sparsity + index map)

Require: Child Jacobian J_x in CSR format; rows to sum: $\{r_0, \ldots, r_{s-1}\}$
Ensure: Output Jacobian J with one row per output element; array `idx_map`

1: *cols* $\leftarrow$ union of column indices across summed rows of J_x
2: Sort *cols*; remove duplicates ▷ $O(\texttt{nnz} \log \texttt{nnz})$
3: Set J.`indptr`, J.`indices` from *cols*
4: Build reverse map `col_to_pos`[ ]: for each unique column c at position p, set `col_to_pos`$[c] \leftarrow p$
5: **for** each nonzero k in J_x (across summed rows) **do**
6: `idx_map`$[k] \leftarrow$ `col_to_pos`$[J_x.\texttt{indices}[k]]$
7: **end for**

Evaluation (Algorithm 2) At each parameter update, `eval_jacobian` zeroes the output values and then scatters the child's nonzeros through the precomputed map. The loop body is a single indexed addition per nonzero—no sorting, no allocation, and no matrix multiply.

Algorithm 2 `sum` — Jacobian evaluation

Require: Child Jacobian J_x (values updated); precomputed `idx_map`
Ensure: Output Jacobian J with accumulated values

1: $J.\texttt{data} \leftarrow \mathbf{0}$
2: **for** $k = 0$ **to** $\texttt{nnz}(J_x) - 1$ **do**
3: $J.\texttt{data}[\texttt{idx_map}[k]]$ += $J_x.\texttt{data}[k]$
4: **end for**

Complexity Initialisation costs $O(\texttt{nnz} \log \texttt{nnz})$ (dominated by the sort) and runs once. Evaluation costs $O(\texttt{nnz})$ with a single pass over the child's nonzeros. While sparse matrix multiplication $\mathbf{1}^T J_x$ is also $O(\texttt{nnz})$ in principle, it must allocate a new result matrix, compute

its sparsity pattern, and sort indices at every call—overhead that the `idx_map` approach eliminates entirely. Figure 3.2 illustrates the CSR transformation for a concrete example.

3.3.3 The `left_matmul` atom and Kronecker avoidance

Left multiplication computes $Y = A\,X$ where $A \in \mathbf{R}^{m\times n}$ is a constant (or parameter) matrix and $X \in \mathbf{R}^{n\times p}$ is the child expression. Writing the operation in vectorised form gives

$$\mathbf{vec}(Y) = (I_p \otimes A)\ \mathbf{vec}(X), \tag{3.1}$$

so the Jacobian with respect to $\mathbf{vec}(X)$ is the block-diagonal matrix $I_p \otimes A$ of size $mp \times np$. Forming this Kronecker product explicitly would require storing $p \cdot \mathtt{nnz}(A)$ entries in the best case; worse, composing it with the child's Jacobian via sparse matrix multiplication is $O(p^2 \cdot \mathtt{nnz}(A))$ in general.

The engine avoids the Kronecker product entirely by exploiting the block-diagonal structure. It stores only A (of size $m \times n$) and applies it independently to each of the p column-blocks of the child's Jacobian. The computation is carried out in CSC (Compressed Sparse Column) format, because the child's Jacobian columns each decompose naturally into p consecutive blocks of n rows.

Initialisation During `jacobian_init`, the child's Jacobian sparsity is converted to CSC. For each column j and each block $b \in \{0, \ldots, p-1\}$, the engine identifies which rows of A have overlapping column support with the child's nonzeros in block b. These overlaps determine the output sparsity without ever forming $I_p \otimes A$.

Evaluation (Algorithm 3) At each parameter update, the child's Jacobian values are converted to CSC, then each output nonzero is computed as a sparse dot product of a row of A with the corresponding block-slice of the child's column.

Complexity Let $\bar{k}$ denote the average number of nonzeros per column of J_{child} within a single block. Each output nonzero requires a sparse dot product of cost at most $O(\mathtt{nnz_row}(A) + \bar{k})$. The total work is therefore $O(\mathtt{nnz}(J) \cdot (\mathtt{nnz_row}(A) + \bar{k}))$, which for typical sparse problems is far smaller than the $O(p^2 \cdot \mathtt{nnz}(A))$ cost of forming and multiplying the Kronecker product.

Algorithm 3 `left_matmul` — Jacobian evaluation (block-diagonal multiply)

Require: Constant matrix $A \in \mathbf{R}^{m \times n}$; child Jacobian J_x in CSC with p column-blocks of n rows; precomputed output sparsity J in CSC
Ensure: Output Jacobian J with filled values

Convert J_x values from CSR to CSC ▷ pattern already fixed
for each column j of J **do**
  for each block $b = 0, \ldots, p-1$ **do**
    $S \leftarrow$ nonzeros of J_x column j in rows $[b \cdot n,\ (b+1) \cdot n)$
    for each output nonzero $(b \cdot m + i,\ j)$ in J **do**
      $J.\texttt{data}[\cdot] \leftarrow \text{sparse_dot}(A[i, :],\ S)$
    end for
  end for
end for
Convert J values from CSC to CSR

3.3.4 The `index` atom and structural row selection

Indexing selects a subset of elements from the child expression: $y = x[\text{indices}]$. Its Jacobian is a row-selection matrix P whose rows are drawn from the identity—row i of P is row indices$[i]$ of I. Since the child's Jacobian is stored in CSR, selecting rows amounts to copying contiguous segments of the `indices` and `data` arrays guided by `indptr`. No arithmetic is performed at all.

Initialisation (Algorithm 4) During `jacobian_init`, the output Jacobian's sparsity pattern is constructed by copying the column-index segment of each selected row from the child's Jacobian. The row pointer array `indptr` is built incrementally.

Algorithm 4 `index` — Jacobian initialisation (structural copy)

Require: Child Jacobian J_x in CSR; selection array $\texttt{idx}[0 \ldots k-1]$
Ensure: Output Jacobian J in CSR with selected rows

$J.\texttt{indptr}[0] \leftarrow 0$
for $i = 0$ **to** $k - 1$ **do**
  $r \leftarrow \texttt{idx}[i]$ ▷ source row in J_x
  $\ell \leftarrow J_x.\texttt{indptr}[r+1] - J_x.\texttt{indptr}[r]$
  Copy $J_x.\texttt{indices}[J_x.\texttt{indptr}[r] \ldots J_x.\texttt{indptr}[r] + \ell)$ into $J.\texttt{indices}[J.\texttt{indptr}[i] \ldots]$
  $J.\texttt{indptr}[i+1] \leftarrow J.\texttt{indptr}[i] + \ell$
end for
$J.\texttt{nnz} \leftarrow J.\texttt{indptr}[k]$

Evaluation (Algorithm 5) At each parameter update, `eval_jacobian` copies the `data` segments for the same selected rows. The sparsity pattern is identical to the child's selected rows, so only values move.

Algorithm 5 `index` — Jacobian evaluation (value copy)

Require: Child Jacobian J_x (values updated); selection array $\texttt{idx}[0 \dots k-1]$; output Jacobian J (pattern fixed)
Ensure: Output Jacobian J with copied values
1: **for** $i = 0$ **to** $k - 1$ **do**
2: $r \leftarrow \texttt{idx}[i]$
3: $\ell \leftarrow J.\texttt{indptr}[i+1] - J.\texttt{indptr}[i]$
4: Copy $J_x.\texttt{data}[J_x.\texttt{indptr}[r] \dots J_x.\texttt{indptr}[r] + \ell)$ into $J.\texttt{data}[J.\texttt{indptr}[i] \dots]$
5: **end for**

Complexity Both initialisation and evaluation cost $O\Big(\sum_{i=0}^{k-1} \texttt{nnz}(\text{row}_{\texttt{idx}[i]})\Big)$, proportional to the total number of nonzeros in the selected rows. The operation involves no arithmetic—only `memcpy` calls—making it one of the cheapest atoms in the engine.

3.4 Optimizations

3.5 Parameter handling

CHAPTER 4 BENCHMARK RESULTS

CHAPTER 5 Benchmark results

CHAPTER 6 CONCLUSION

Texte / Text.

6.1 Synthèse des travaux / Summary of Works

Texte / Text.

6.2 Limitations de la solution proposée / Limitations

6.3 Améliorations futures / Future Research

Texte / Text.

REFERENCES

[1] R. Fourer and M. J. Harrison, "A modern approach to computer systems for linear programming," 1978.

[2] *IBM Mathematical Programming System Extended/370 (MPSX/370) Program Reference Manual*, 2nd ed., IBM Corporation, 1976.

[3] W. Orchard-Hays, "History of mathematical programming systems," *Annals of the History of Computing*, vol. 6, no. 3, pp. 296–312, 1984.

[4] I. Marceau and T. W. Mason, "A matrix-generator system for linear programming problems," Center for Advanced Computation, University of Illinois at Urbana-Champaign, Urbana, IL, Tech. Rep. CAC Document No. 4, 1971. [Online]. Available: https://hdl.handle.net/2142/32405

[5] *MaGen Reference Manual*, Haverly Systems, Inc., Denville, NJ, 1977.

[6] *GAMMA 3.4 Programmer Reference*, Sperry Univac Computer Systems, St. Paul, MN, 1977.

[7] *MIPS III DATAFORM User Manual*, Ketron, Inc., Arlington, VA, 1975.

[8] R. Fourer, "Modeling languages versus matrix generators for linear programming," *ACM Trans. Math. Softw.*, vol. 9, no. 2, pp. 143–183, Jun. 1983. [Online]. Available: https://doi.org/10.1145/357456.357457

[9] A. Brooke, D. Kendrick, and A. Meeraus, *GAMS: A User's Guide*. Redwood City, CA: The Scientific Press, 1988.

[10] J. Bisschop, *AIMMS optimization modeling*, 2006.

[11] R. Fourer, D. M. Gay, and B. W. Kernighan, "A modeling language for mathematical programming," *Management Science*, vol. 36, no. 5, pp. 519–554, 1990.

[12] M. ApS, *MOSEK optimization suite 11.0*, 2026.

[13] ——, "MOSEK modeling cookbook," 2026.

[14] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[15] J. Löfberg, "YALMIP: A toolbox for modeling and optimization in MATLAB," in *2004 IEEE international conference on robotics and automation (IEEE Cat. No. 04CH37508)*. IEEE, 2004, pp. 284–289.

[16] W. E. Hart, J.-P. Watson, and D. L. Woodruff, "Pyomo: modeling and solving mathematical programs in Python," *Mathematical Programming Computation*, vol. 3, no. 3, pp. 219–260, 2011.

[17] I. Dunning, J. Huchette, and M. Lubin, "JuMP: A modeling language for mathematical optimization," *SIAM Review*, vol. 59, no. 2, pp. 295–320, 2017.

[18] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[19] M. Grant, "Disciplined Convex Programming," Ph.D. dissertation, Stanford University, Dec. 2004, ph.D. dissertation. [Online]. Available: https://web.stanford.edu/~boyd/papers/pdf/mcg_thesis.pdf

[20] M. Grant, S. Boyd, and Y. Ye, "Disciplined convex programming," in *Global Optimization: From Theory to Implementation*, L. Liberti and N. Maculan, Eds. New York: Springer, 2006, pp. 155–210.

[21] I. CVX Research, "CVX: Matlab software for disciplined convex programming, version 2.0," https://cvxr.com/cvx, Aug. 2012.

[22] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[23] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, "A rewriting system for convex optimization problems," *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

[24] A. Fu, B. Narasimhan, and S. Boyd, "Cvxr: An r package for disciplined convex optimization," *Journal of Statistical Software*, vol. 94, no. 14, 2020. [Online]. Available: http://dx.doi.org/10.18637/jss.v094.i14

[25] M. Udell, K. Mohan, D. Zeng, J. Hong, S. Diamond, and S. Boyd, "Convex optimization in Julia," in *2014 first workshop for high performance technical computing in dynamic languages*. IEEE, 2014, pp. 18–28.

[26] M. Lubin, O. Dowson, J. D. Garcia, J. Huchette, B. Legat, and J. P. Vielma, "Jump 1.0: Recent improvements to a modeling language for mathematical optimization," 2023. [Online]. Available: https://arxiv.org/abs/2206.03866

[27] B. Legat, O. Dowson, J. D. Garcia, and M. Lubin, "Mathoptinterface: a data structure for mathematical optimization problems," 2020. [Online]. Available: https://arxiv.org/abs/2002.03447

[28] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.

[29] C. Coey, L. Kapelevich, and J. P. Vielma, "Solving natural conic formulations with Hypatia.jl," *INFORMS Journal on Computing*, vol. 34, no. 5, pp. 2686–2699, 2022.

[30] ——, "Performance enhancements for a generic conic interior point algorithm," *Mathematical Programming Computation*, vol. 15, no. 1, pp. 53–101, 2022.

[31] B. O'Donoghue, E. Chu, N. Parikh, and S. Boyd, "Conic optimization via operator splitting and homogeneous self-dual embedding," *Journal of Optimization Theory and Applications*, vol. 169, no. 3, pp. 1042–1068, June 2016. [Online]. Available: http://stanford.edu/~boyd/papers/scs.html

[32] B. O'Donoghue, "Operator splitting for a homogeneous embedding of the linear complementarity problem," *SIAM Journal on Optimization*, vol. 31, pp. 1999–2023, August 2021.

[33] D. Cederberg and S. Boyd, "Projections onto spectral matrix cones," 2025. [Online]. Available: https://arxiv.org/abs/2511.01089

[34] A. Agrawal, S. Diamond, and S. Boyd, "Disciplined geometric programming," *Optimization Letters*, vol. 13, pp. 961–976, 2019.

[35] A. Agrawal and S. Boyd, "Disciplined quasiconvex programming," *Optimization Letters*, vol. 14, pp. 1643–1657, 2020.

[36] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and Z. Kolter, "Differentiable convex optimization layers," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[37] M. Schaller, G. Banjac, S. Diamond, A. Agrawal, B. Stellato, and S. Boyd, "Embedded code generation with cvxpy," *IEEE Control Systems Letters*, vol. 6, pp. 2653–2658, 2022.

[38] E. Chu, N. Parikh, A. Domahidi, and S. Boyd, "Code generation for embedded second-order cone programming," in *2013 European Control Conference (ECC)*, 2013, pp. 1547–1552.

[39] Gurobi Optimization, LLC, *Gurobi Optimizer Reference Manual*, 2026. [Online]. Available: https://www.gurobi.com

[40] A. Domahidi, E. Chu, and S. Boyd, "ECOS: An SOCP solver for embedded systems," in *European Control Conference (ECC)*, 2013, pp. 3071–3076.

APPENDIX A DÉMO

Texte de l'annexe A. Remarquez que la phrase précédente se termine par une lettre majuscule suivie d'un point. On indique explicitement cette situation à LaTeX afin que ce dernier ajuste correctement l'espacement entre le point final de la phrase et le début de la phrase suivante.

APPENDIX B ENCORE UNE ANNEXE / ANOTHER APPENDIX

Texte de l'annexe B en mode «landscape».

APPENDIX C UNE DERNIÈRE ANNEXE / THE LAST APPENDIX

Texte de l'annexe C.